

VYNL: Complete Backend Flowcharts, Revision 2

First-principles re-architecture with all previously open gaps resolved

Architecture rule (unchanged): FastAPI → domain service → Supabase / Telegram or object store / external providers / configured LLM API. No VYNL-owned recommendation model is trained for MVP.

What changed from Revision 1

- Every request flows through a fixed layered path: middleware → handler → request context → service → repository / ports.
- Anything slow or third-party (song acquisition, enrichment, LLM calls, playlist generation, Monthly Wrap, account deletion) is now an asynchronous job. The API returns 202 and the client polls or receives SSE.
- Redis is added for the job queue (Streams), cache, pub/sub for collaboration, and rate limits. The deployment stays a modular monolith with two process types (api, worker).
- Audio bytes never pass through business endpoints. Delivery is by presigned object-storage URL (recommended) or a signed streaming endpoint with Range support.
- Each flow page now ends with “Gaps filled” (what Revision 1 left open, and the decision taken) and “Still open” (what needs a human owner).
- Two flows are new: 19 (Background Jobs & Scheduler) and 20 (Graceful Shutdown & Deploy).
- Decisions marked proposed are this revision’s defaults, not source-document requirements. They should be confirmed by the product owner.

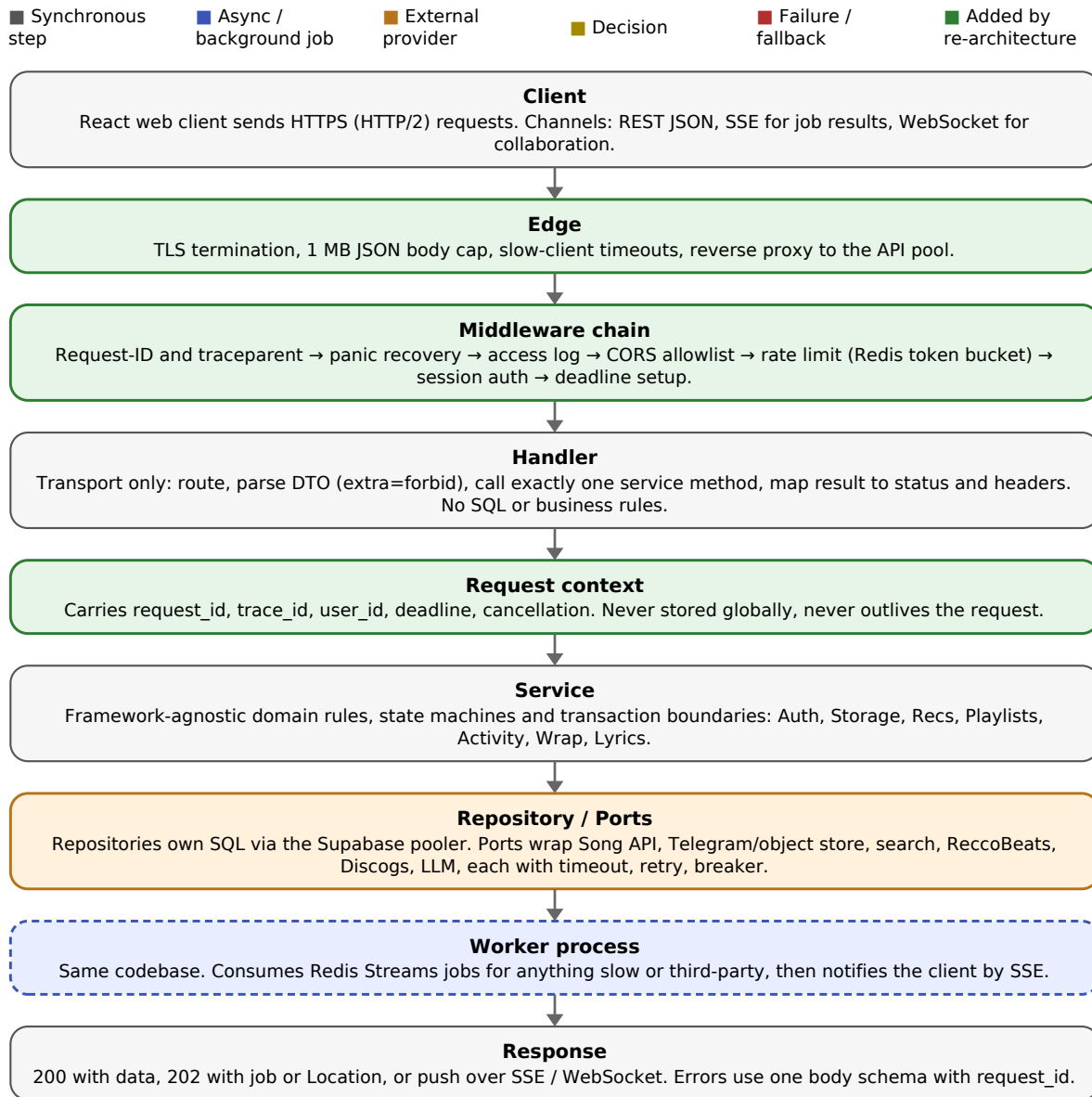
Decision register (Revision 1 gap → Revision 2 decision)

Area	Revision 1 gap	Revision 2 decision
Login (F1)	Local credentials vs Google OAuth unresolved	Google OAuth 2.0 only for MVP (Authorization Code + PKCE). Opaque server-side sessions. If local passwords return later: Argon2id.
Search (F2)	No search provider or endpoint defined	SearchPort with a normalized SongDTO. Proposed default adapter: the Song API search if it has one, else MusicBrainz (about 1 req/s). Cached 5 min.
Audio acquisition (F3)	Fetch ran inside the request; duplicate fetches possible	Song state machine NEW / ACQUIRING / READY / FAILED. Unique identity_key. One acquire_song job per song. API returns 202.
Audio delivery (F3, F12)	“Temporary link from File ID” undefined	Preferred: promote audio to object storage and issue presigned URLs, Telegram kept as cold backup. If Telegram-only: HMAC-signed stream endpoint, 10 min expiry, Range support.
Events (F5, F13)	Delivery mechanism and buffering unspecified	Client batches up to 100 events or 15 s, with a UUID event_id. POST /activity/batch is idempotent. Monthly partitions.
Recommendation (F6)	Prompt size, count, ranking open	Token-budgeted context, 10 recommendations by default, JSON schema validated, order = LLM order after filtering. Job-based with a 12 s resolve deadline.
AI playlist (F8)	Max size unspecified	Target size 10–50, default 20. Draft held 1 h in Redis, persisted only on save.
Playlist edits (F9)	Edit endpoints unspecified	PATCH / PUT with If-Match version (optimistic lock). Fractional-index positions.
Collaboration (F10)	AI trigger frequency and invitation mechanism open	Invite = signed one-time token (24 h). AI suggestion every 10 playlist versions or after 5 min idle, coalesced. Redis pub/sub fan-out.
Lyrics (F11)	Provider, timestamp format, image storage open	LyricsPort. Proposed default: LRCLIB, LRC timestamps parsed to [{t_ms, text}]. Backdrops go to object storage via presigned PUT (2 MB, png/jpeg/webp).
Monthly Wrap (F14)	“Newly discovered artist” and formulas undefined	Play counts if listened ≥30 s. Replay = song with ≥2 counted plays. Skip = explicit skip or <30 s. New artist = 0 plays in the prior 12 months, ≥1 in period.
LLM fallback (F15)	Deterministic policy not finalized	Ladder: unplayed queued → liked and not played in 24 h → nearest by audio features → leave queue untouched.

Area	Revision 1 gap	Revision 2 decision
Retries (F15, F16)	Retry strategy open	Exponential backoff with jitter. Enrichment retried at 1 h / 6 h / 24 h, max 5. Breaker: 5 failures or 50% errors over 30 s.
Deletion (F17)	Cascade and retention undefined	Async delete_account job: hard-delete personal rows, anonymize shared records, keep songs. Completed within 30 days.
Observability (F18)	Metrics backend and thresholds unspecified	Structured JSON logs, OpenTelemetry traces, Prometheus metrics, starter alert thresholds defined in flow 18.
Jobs, shutdown (new)	Not covered	Flows 19 and 20 define queue semantics, scheduler, and SIGTERM drain protocol.

0. Master Backend Request & Service Map

Every feature enters through FastAPI, passes the same middleware chain, reaches one service, and touches Postgres, Redis, or an external provider only through a port.



Backend execution

1. **Client:** React web client sends HTTPS (HTTP/2) requests. Channels: REST JSON, SSE for job results, WebSocket for collaboration.
2. **Edge:** TLS termination, 1 MB JSON body cap, slow-client timeouts, reverse proxy to the API pool.
3. **Middleware chain:** Request-ID and traceparent → panic recovery → access log → CORS allowlist → rate limit (Redis token bucket) → session auth → deadline setup.
4. **Handler:** Transport only: route, parse DTO (extra=forbid), call exactly one service method, map result to status and headers. No SQL or business rules.
5. **Request context:** Carries request_id, trace_id, user_id, deadline, cancellation. Never stored globally, never outlives the request.
6. **Service:** Framework-agnostic domain rules, state machines and transaction boundaries: Auth, Storage, Recs, Playlists, Activity, Wrap, Lyrics.
7. **Repository / Ports:** Repositories own SQL via the Supabase pooler. Ports wrap Song API, Telegram/object store, search, ReccoBeats, Discogs, LLM, each with timeout, retry, breaker.

8. **Worker process:** Same codebase. Consumes Redis Streams jobs for anything slow or third-party, then notifies the client by SSE.

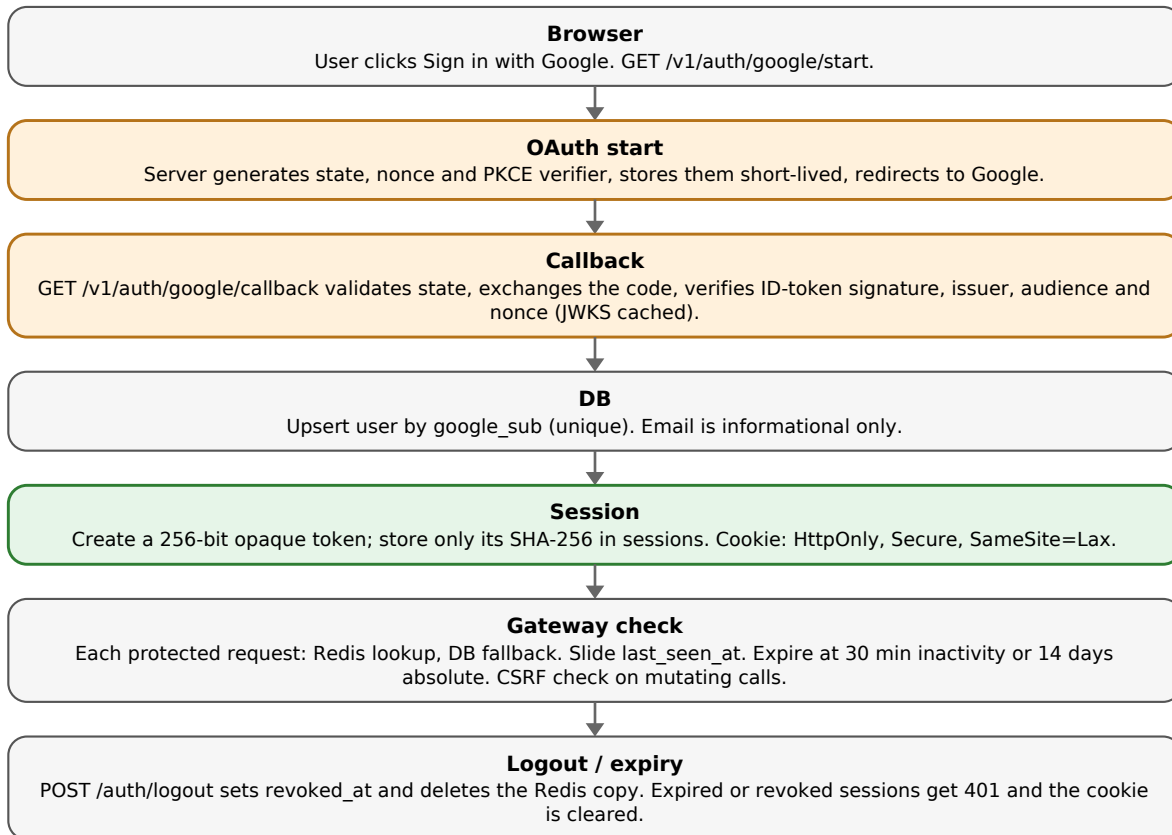
9. **Response:** 200 with data, 202 with job or Location, or push over SSE / WebSocket. Errors use one body schema with request_id.

Gaps filled (decisions in this revision)

- Deployment is a modular monolith with two process types: api and worker, one image.
- Redis is added for queue, cache, pub/sub and rate limits.
- Modules communicate only through service interfaces, never through each other's tables.

1. User Registration, Google OAuth & Session Management

Authentication gates search, playback, playlists and Monthly Wrap.



Backend execution

1. **Browser**: User clicks Sign in with Google. GET /v1/auth/google/start.
2. **OAuth start**: Server generates state, nonce and PKCE verifier, stores them short-lived, redirects to Google.
3. **Callback**: GET /v1/auth/google/callback validates state, exchanges the code, verifies ID-token signature, issuer, audience and nonce (JWKS cached).
4. **DB**: Upsert user by google_sub (unique). Email is informational only.
5. **Session**: Create a 256-bit opaque token; store only its SHA-256 in sessions. Cookie: HttpOnly, Secure, SameSite=Lax.
6. **Gateway check**: Each protected request: Redis lookup, DB fallback. Slide last_seen_at. Expire at 30 min inactivity or 14 days absolute. CSRF check on mutating calls.
7. **Logout / expiry**: POST /auth/logout sets revoked_at and deletes the Redis copy. Expired or revoked sessions get 401 and the cookie is cleared.

Gaps filled (decisions in this revision)

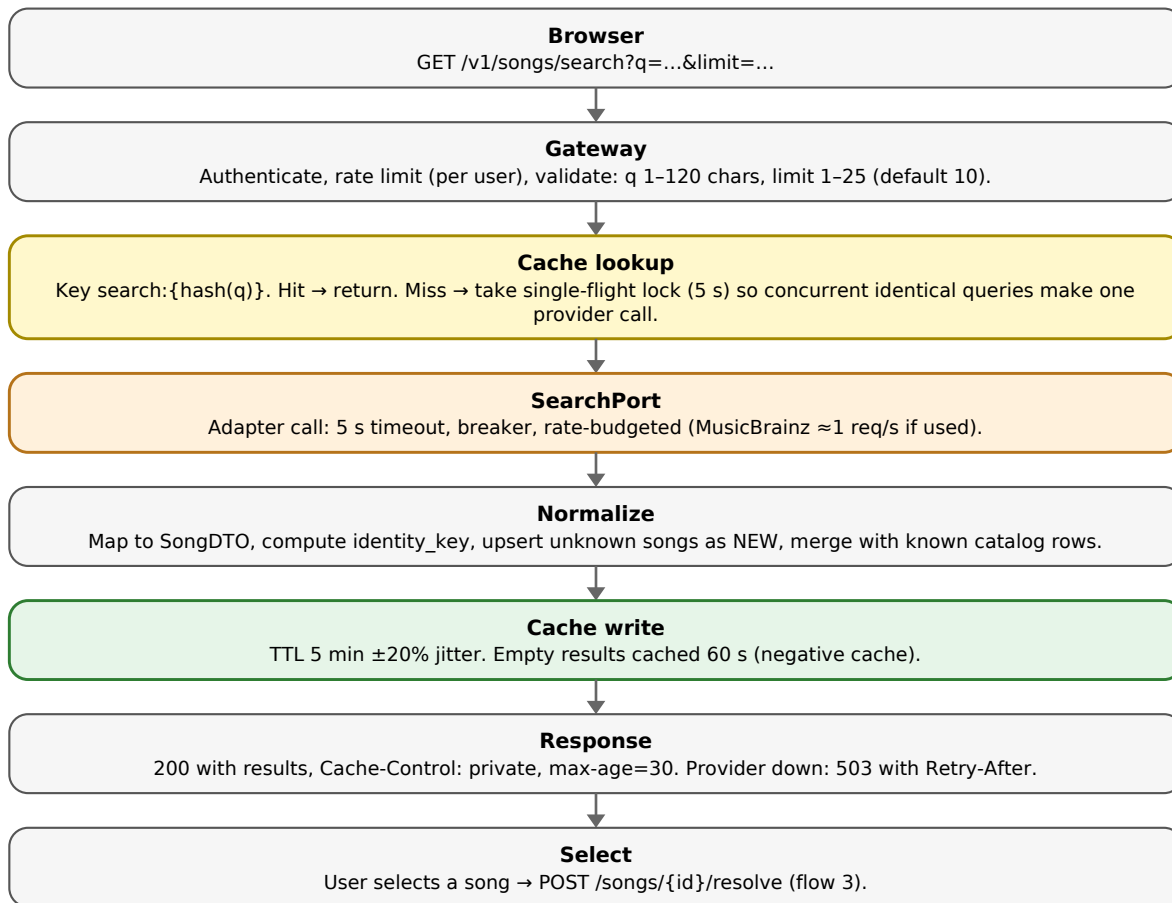
- MVP uses Google OAuth only, which resolves the local-credentials question. If local passwords are added later: Argon2id, per-account lockout, never log passwords.
- Inactivity expiry and logout invalidation work because sessions are server-side, not self-contained JWTs.
- Mutating cookie-authenticated requests need X-CSRF-Token (double-submit) plus an Origin check.

Still open

- Whether to add email/password later is a product decision.

2. Song Search

Search is an authenticated, cacheable read. Selecting a result enters the resolve pipeline.



Backend execution

1. **Browser**: GET /v1/songs/search?q=...&limit=...
2. **Gateway**: Authenticate, rate limit (per user), validate: q 1-120 chars, limit 1-25 (default 10).
3. **Cache lookup**: Key search:{hash(q)}. Hit → return. Miss → take single-flight lock (5 s) so concurrent identical queries make one provider call.
4. **SearchPort**: Adapter call: 5 s timeout, breaker, rate-budgeted (MusicBrainz ≈1 req/s if used).
5. **Normalize**: Map to SongDTO, compute identity_key, upsert unknown songs as NEW, merge with known catalog rows.
6. **Cache write**: TTL 5 min ±20% jitter. Empty results cached 60 s (negative cache).
7. **Response**: 200 with results, Cache-Control: private, max-age=30. Provider down: 503 with Retry-After.
8. **Select**: User selects a song → POST /songs/{id}/resolve (flow 3).

Gaps filled (decisions in this revision)

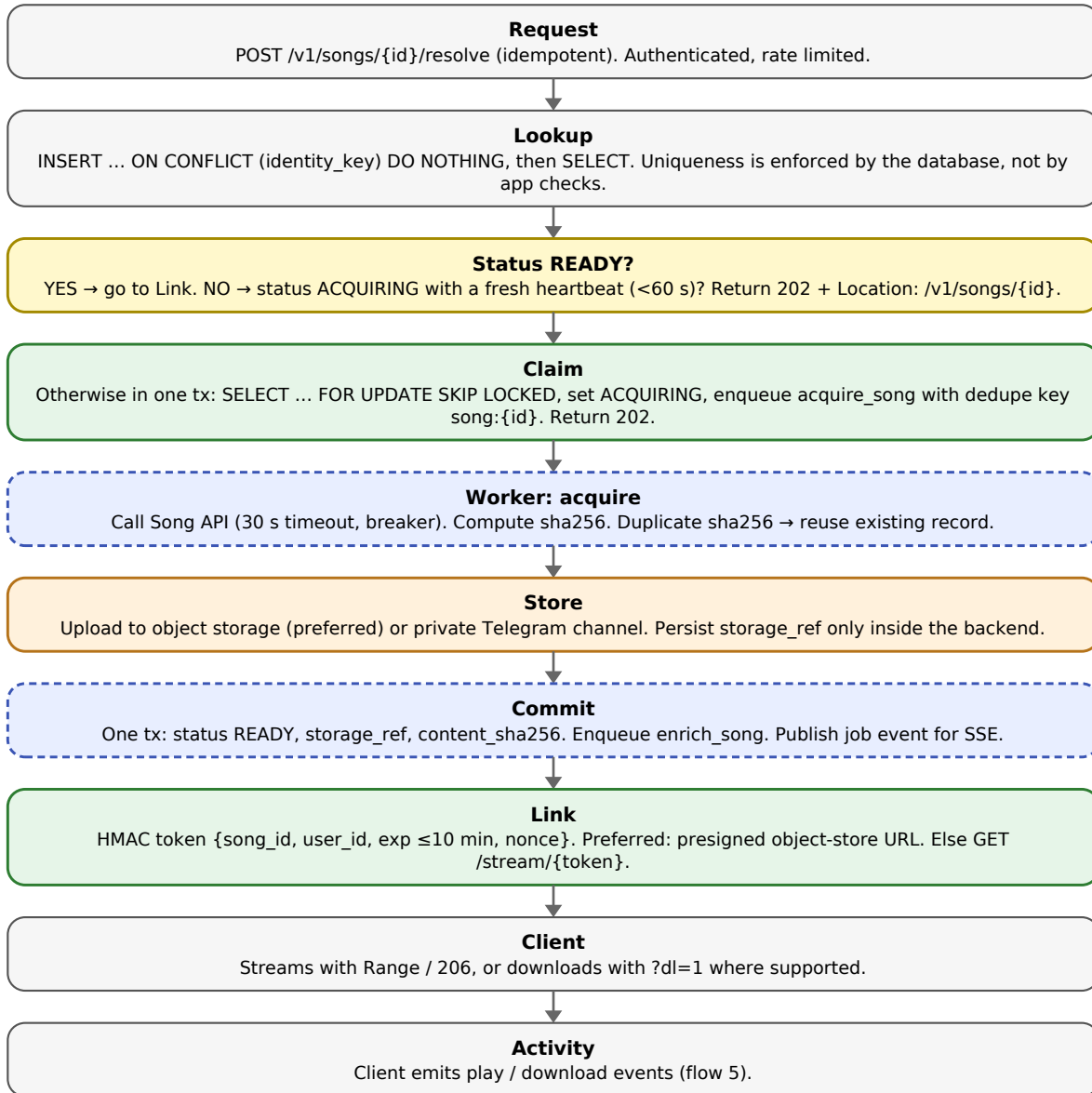
- A SearchPort contract now exists, so the provider is replaceable. The Song API is not assumed to be the search provider.
- Search cannot stampede or be hit with unbounded queries: jitter, negative caching, single-flight and rate limits are in place.

Still open

- The concrete search provider must be confirmed (see Appendix C).

3. Fetch-Once-and-Reuse Storage, Playback & Download

The core audio path. READY songs return a link at once. Misses become one background acquisition shared by all requesters.



Backend execution

1. **Request:** POST /v1/songs/{id}/resolve (idempotent). Authenticated, rate limited.
2. **Lookup:** INSERT ... ON CONFLICT (identity_key) DO NOTHING, then SELECT. Uniqueness is enforced by the database, not by app checks.
3. **Status READY?:** YES → go to Link. NO → status ACQUIRING with a fresh heartbeat (<60 s)? Return 202 + Location: /v1/songs/{id}.
4. **Claim:** Otherwise in one tx: SELECT ... FOR UPDATE SKIP LOCKED, set ACQUIRING, enqueue acquire_song with dedupe key song:{id}. Return 202.
5. **Worker: acquire:** Call Song API (30 s timeout, breaker). Compute sha256. Duplicate sha256 → reuse existing record.
6. **Store:** Upload to object storage (preferred) or private Telegram channel. Persist storage_ref only inside the backend.
7. **Commit:** One tx: status READY, storage_ref, content_sha256. Enqueue enrich_song. Publish job event for SSE.
8. **Link:** HMAC token {song_id, user_id, exp ≤10 min, nonce}. Preferred: presigned object-store URL. Else GET /stream/{token}.
9. **Client:** Streams with Range / 206, or downloads with ?dl=1 where supported.
10. **Activity:** Client emits play / download events (flow 5).

Gaps filled (decisions in this revision)

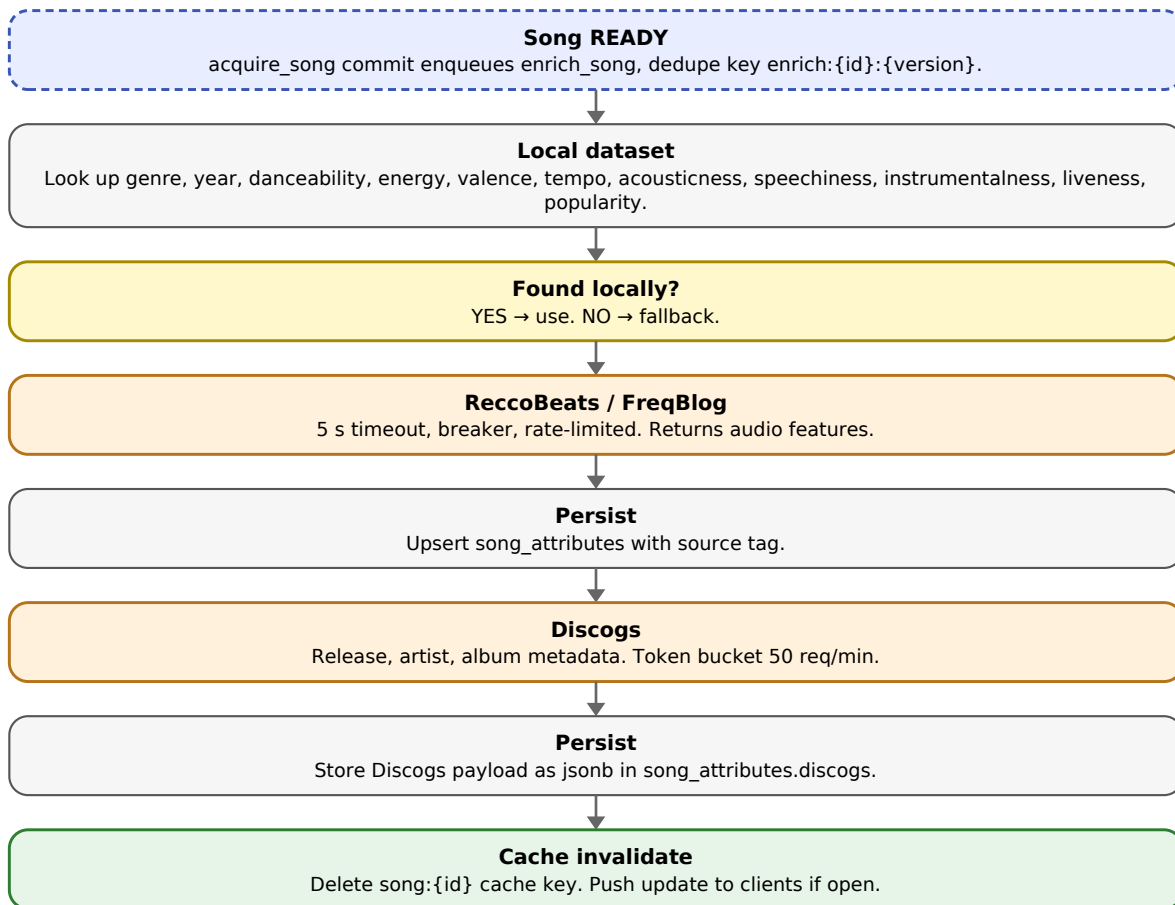
- Targets: READY playback starts in <3 s. First acquisition returns 202 immediately, so the <10 s target applies to the worker, excluding third-party latency.
- Failure policy: 4 retries, exponential backoff with jitter, then status FAILED and DLQ. A reaper requeues rows stuck in ACQUIRING over 2 min.
- Storage_ref and Telegram file_id are never sent to the client. The Telegram bot token is never part of any URL.
- Telegram's Bot API cannot sign URLs and caps getFile at 20 MB. Therefore audio is promoted to object storage with presigned URLs, and Telegram is the backup copy. A Telegram-only deployment must use the signed streaming endpoint (async generator, no full buffering).

Still open

- Object store vs Telegram-only: confirm. Legal review of audio sourcing and Telegram storage: required before launch.

4. Metadata, Audio-Feature & Discogs Enrichment

Enrichment is a background job off the critical path. Playback never waits for it.



Backend execution

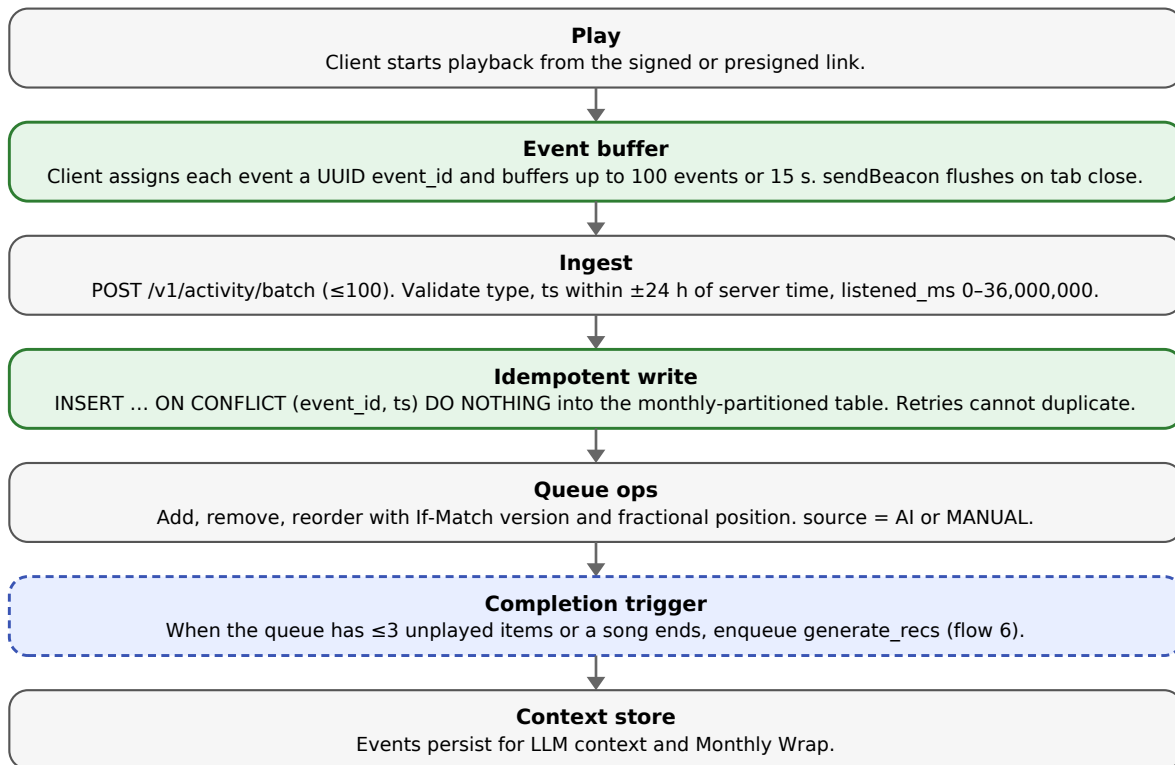
1. **Song READY:** acquire_song commit enqueues enrich_song, dedupe key enrich:{id}:{version}.
2. **Local dataset:** Look up genre, year, danceability, energy, valence, tempo, acousticness, speechiness, instrumentalness, liveness, popularity.
3. **Found locally?:** YES → use. NO → fallback.
4. **ReccoBeats / FreqBlog:** 5 s timeout, breaker, rate-limited. Returns audio features.
5. **Persist:** Upsert song_attributes with source tag.
6. **Discogs:** Release, artist, album metadata. Token bucket 50 req/min.
7. **Persist:** Store Discogs payload as jsonb in song_attributes.discogs.
8. **Cache invalidate:** Delete song:{id} cache key. Push update to clients if open.

Gaps filled (decisions in this revision)

- Enrichment is asynchronous with explicit rate limits and a breaker, so a slow provider cannot delay a stream.
- On failure the schedule is 1 h / 6 h / 24 h, max 5 attempts, then enrichment_failed is flagged (see flow 16).

5. Playback Session, Queue & Activity Logging

The runtime loop after a song resolves, and how behavior becomes future context.



Backend execution

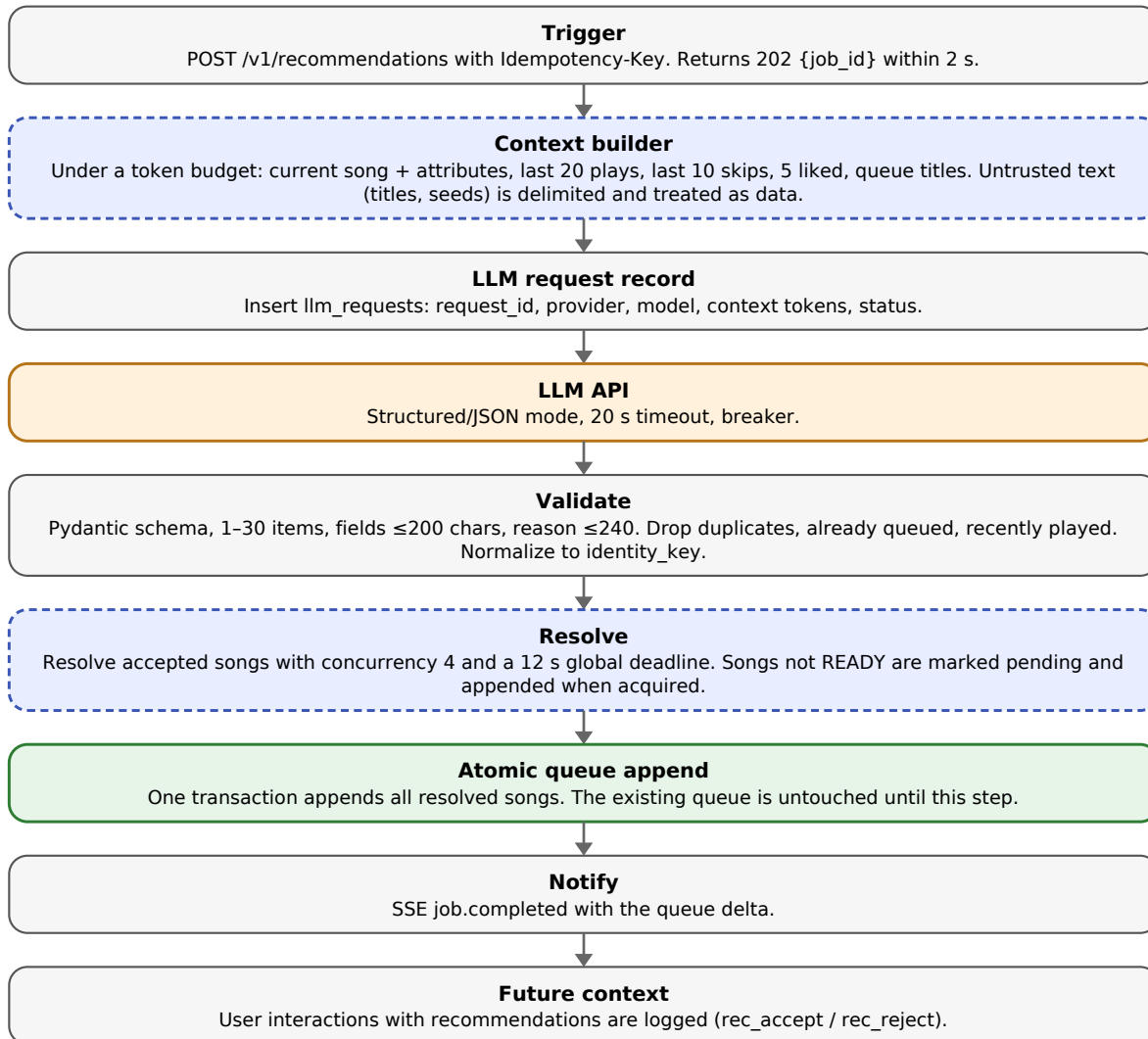
1. **Play**: Client starts playback from the signed or presigned link.
2. **Event buffer**: Client assigns each event a UUID event_id and buffers up to 100 events or 15 s. sendBeacon flushes on tab close.
3. **Ingest**: POST /v1/activity/batch (≤100). Validate type, ts within ±24 h of server time, listened_ms 0-36,000,000.
4. **Idempotent write**: INSERT ... ON CONFLICT (event_id, ts) DO NOTHING into the monthly-partitioned table. Retries cannot duplicate.
5. **Queue ops**: Add, remove, reorder with If-Match version and fractional position. source = AI or MANUAL.
6. **Completion trigger**: When the queue has ≤3 unplayed items or a song ends, enqueue generate_recs (flow 6).
7. **Context store**: Events persist for LLM context and Monthly Wrap.

Gaps filled (decisions in this revision)

- Event delivery is now defined: client batching with idempotent server ingest.
- Queue writes use optimistic locking, so two tabs cannot corrupt order.

6. AI Next-Song Recommendation: LLM Runtime

The LLM API is the decision layer. The backend owns context, validation, resolution and atomic queue commit.



Backend execution

1. **Trigger**: POST /v1/recommendations with Idempotency-Key. Returns 202 {job_id} within 2 s.
2. **Context builder**: Under a token budget: current song + attributes, last 20 plays, last 10 skips, 5 liked, queue titles. Untrusted text (titles, seeds) is delimited and treated as data.
3. **LLM request record**: Insert llm_requests: request_id, provider, model, context tokens, status.
4. **LLM API**: Structured/JSON mode, 20 s timeout, breaker.
5. **Validate**: Pydantic schema, 1-30 items, fields ≤ 200 chars, reason ≤ 240 . Drop duplicates, already queued, recently played. Normalize to identity_key.
6. **Resolve**: Resolve accepted songs with concurrency 4 and a 12 s global deadline. Songs not READY are marked pending and appended when acquired.
7. **Atomic queue append**: One transaction appends all resolved songs. The existing queue is untouched until this step.
8. **Notify**: SSE job.completed with the queue delta.
9. **Future context**: User interactions with recommendations are logged (rec_accept / rec_reject).

Gaps filled (decisions in this revision)

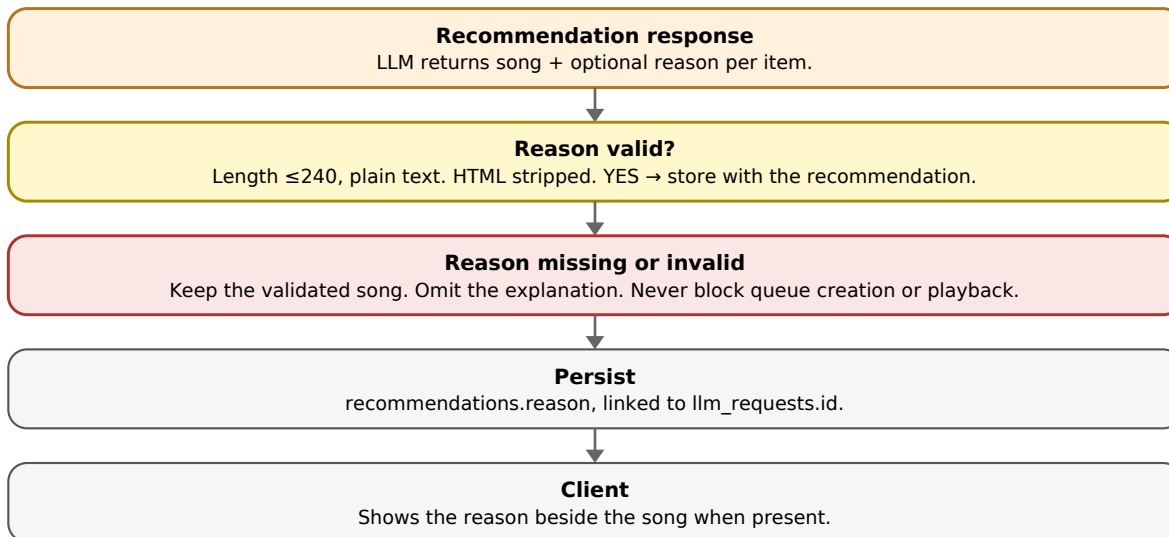
- Default is 10 recommendations. Ranking is the LLM's order after filtering. No VYNL-trained model exists.
- Prompt-injection guard: untrusted strings are delimited and the output is accepted only if it parses against the schema.
- Prompt text itself is not logged, only a hash and token counts.

Still open

- Provider and model choice.

7. Recommendation Reasoning / Explanation

Reasons are returned in the same LLM call. A separate explanation call is not required.



Backend execution

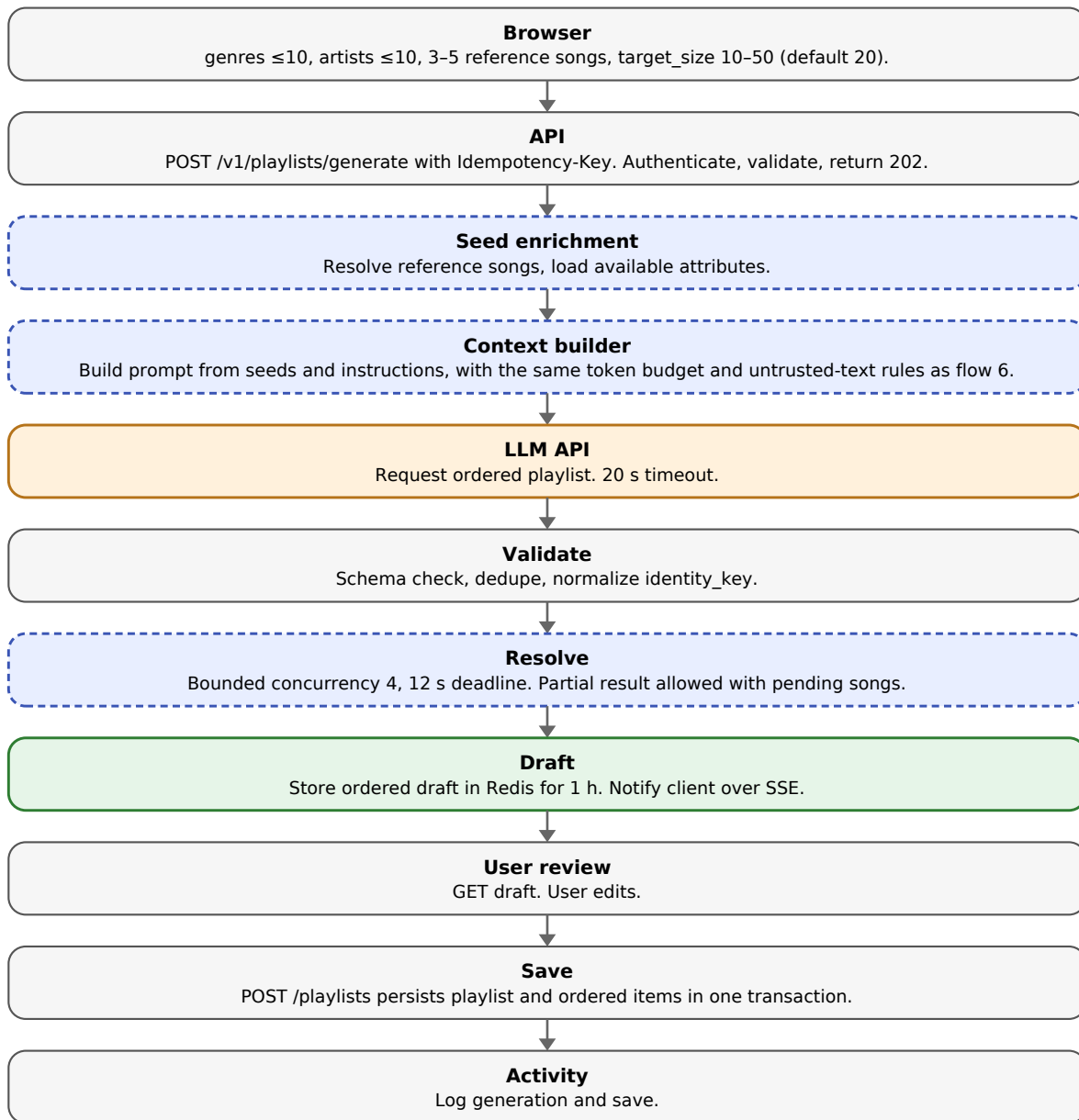
1. **Recommendation response:** LLM returns song + optional reason per item.
2. **Reason valid?:** Length ≤ 240 , plain text. HTML stripped. YES → store with the recommendation.
3. **Reason missing or invalid:** Keep the validated song. Omit the explanation. Never block queue creation or playback.
4. **Persist:** recommendations.reason, linked to llm_requests.id.
5. **Client:** Shows the reason beside the song when present.

Gaps filled (decisions in this revision)

- Reason length and sanitization rules are defined. NFR-3.3 (show recommendations without explanation) is implemented as the fallback path.

8. AI-Assisted Playlist Generation

Seeds become an LLM job. The draft is not persisted until the user saves it.



Backend execution

1. **Browser**: genres ≤10, artists ≤10, 3-5 reference songs, target_size 10-50 (default 20).
2. **API**: POST /v1/playlists/generate with Idempotency-Key. Authenticate, validate, return 202.
3. **Seed enrichment**: Resolve reference songs, load available attributes.
4. **Context builder**: Build prompt from seeds and instructions, with the same token budget and untrusted-text rules as flow 6.
5. **LLM API**: Request ordered playlist. 20 s timeout.
6. **Validate**: Schema check, dedupe, normalize identity_key.
7. **Resolve**: Bounded concurrency 4, 12 s deadline. Partial result allowed with pending songs.
8. **Draft**: Store ordered draft in Redis for 1 h. Notify client over SSE.
9. **User review**: GET draft. User edits.
10. **Save**: POST /playlists persists playlist and ordered items in one transaction.
11. **Activity**: Log generation and save.

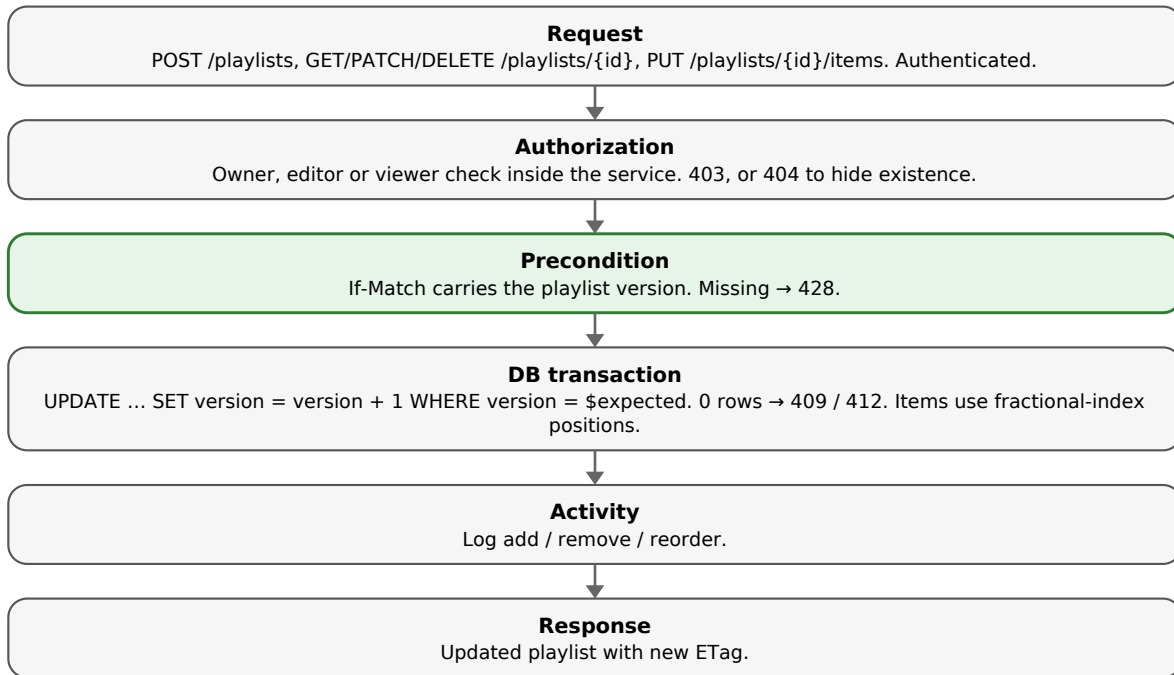
Gaps filled (decisions in this revision)

- NFR-1.4 (15 s) is met by a 12 s resolve deadline with partial results. Maximum size is 50.

- Drafts expire; nothing is stored in the playlist tables until the user saves.

9. Playlist Creation, Editing & Saving

Non-AI playlist management with optimistic concurrency.



Backend execution

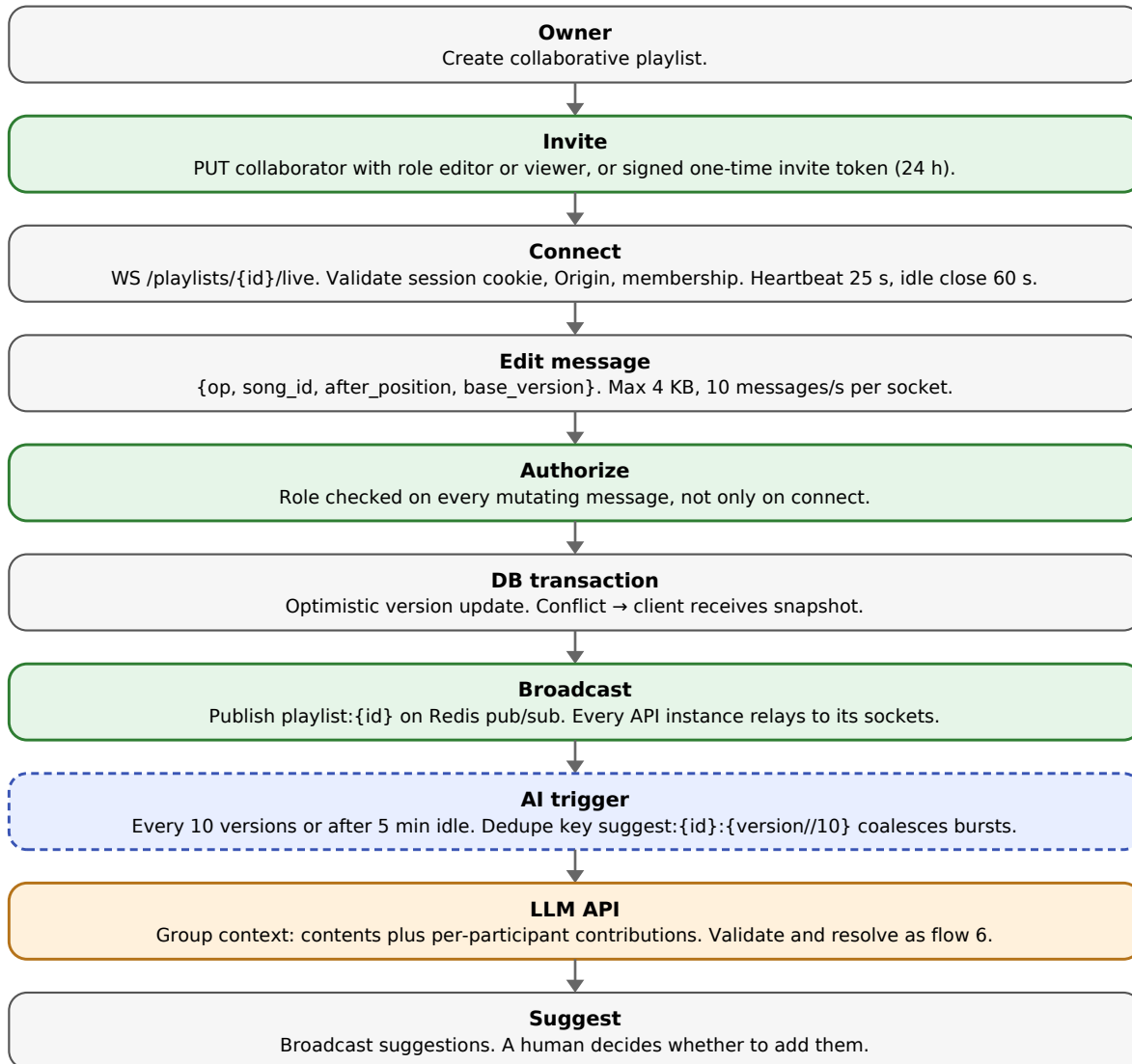
1. **Request:** POST /playlists, GET/PATCH/DELETE /playlists/{id}, PUT /playlists/{id}/items. Authenticated.
2. **Authorization:** Owner, editor or viewer check inside the service. 403, or 404 to hide existence.
3. **Precondition:** If-Match carries the playlist version. Missing → 428.
4. **DB transaction:** UPDATE ... SET version = version + 1 WHERE version = \$expected. 0 rows → 409 / 412. Items use fractional-index positions.
5. **Activity:** Log add / remove / reorder.
6. **Response:** Updated playlist with new ETag.

Gaps filled (decisions in this revision)

- Edit endpoints and concurrency control are specified. A reorder moves one row's position key instead of rewriting the list.

10. Collaborative Playlist + Real-Time AI Suggestions

Many users edit one playlist over WebSocket. Changes are serialized by the database and fanned out via Redis.



Backend execution

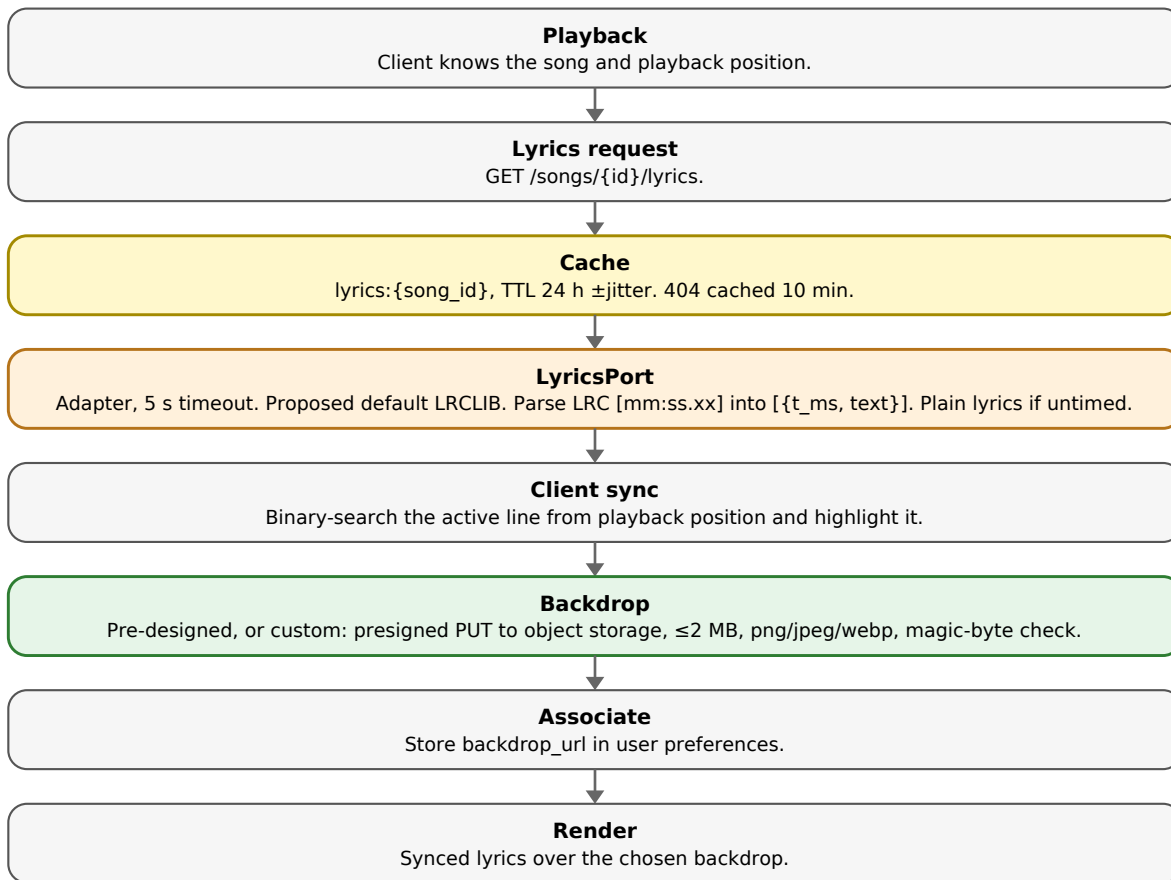
1. **Owner:** Create collaborative playlist.
2. **Invite:** PUT collaborator with role editor or viewer, or signed one-time invite token (24 h).
3. **Connect:** WS /playlists/{id}/live. Validate session cookie, Origin, membership. Heartbeat 25 s, idle close 60 s.
4. **Edit message:** {op, song_id, after_position, base_version}. Max 4 KB, 10 messages/s per socket.
5. **Authorize:** Role checked on every mutating message, not only on connect.
6. **DB transaction:** Optimistic version update. Conflict → client receives snapshot.
7. **Broadcast:** Publish playlist:{id} on Redis pub/sub. Every API instance relays to its sockets.
8. **AI trigger:** Every 10 versions or after 5 min idle. Dedupe key suggest:{id}:{version//10} coalesces bursts.
9. **LLM API:** Group context: contents plus per-participant contributions. Validate and resolve as flow 6.
10. **Suggest:** Broadcast suggestions. A human decides whether to add them.

Gaps filled (decisions in this revision)

- Invitation mechanism and AI trigger frequency are decided. Max 50 sockets per playlist.
- Close codes: 1001 on server shutdown, 4403 when permission is revoked mid-session.

11. Synchronized Interactive Lyrics

Lyrics are provider-backed data synced on the client. Backdrops are stored in object storage.



Backend execution

1. **Playback:** Client knows the song and playback position.
2. **Lyrics request:** GET /songs/{id}/lyrics.
3. **Cache:** lyrics:{song_id}, TTL 24 h ± jitter. 404 cached 10 min.
4. **LyricsPort:** Adapter, 5 s timeout. Proposed default LRCLIB. Parse LRC [mm:ss.xx] into [{t_ms, text}]. Plain lyrics if untimed.
5. **Client sync:** Binary-search the active line from playback position and highlight it.
6. **Backdrop:** Pre-designed, or custom: presigned PUT to object storage, ≤2 MB, png/jpeg/webp, magic-byte check.
7. **Associate:** Store backdrop_url in user preferences.
8. **Render:** Synced lyrics over the chosen backdrop.

Gaps filled (decisions in this revision)

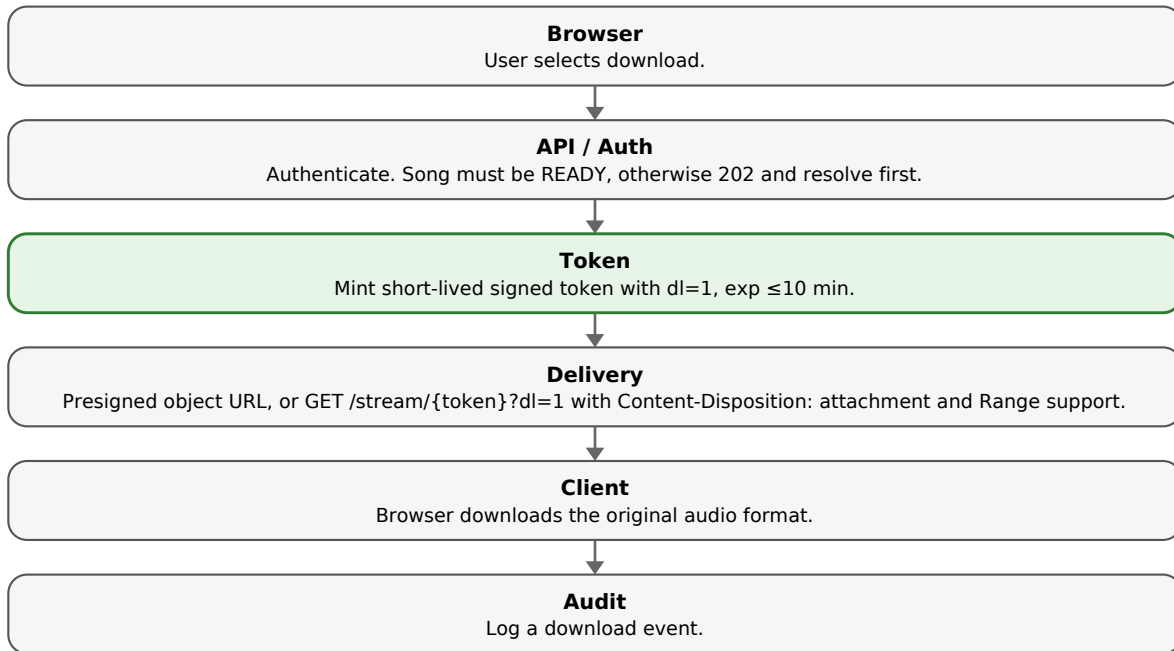
- Timestamp format is LRC, normalized server-side to milliseconds. Image bytes never pass through FastAPI.

Still open

- Lyrics provider licensing terms must be confirmed.

12. Song Download

Download reuses the stored audio. No second copy is acquired.



Backend execution

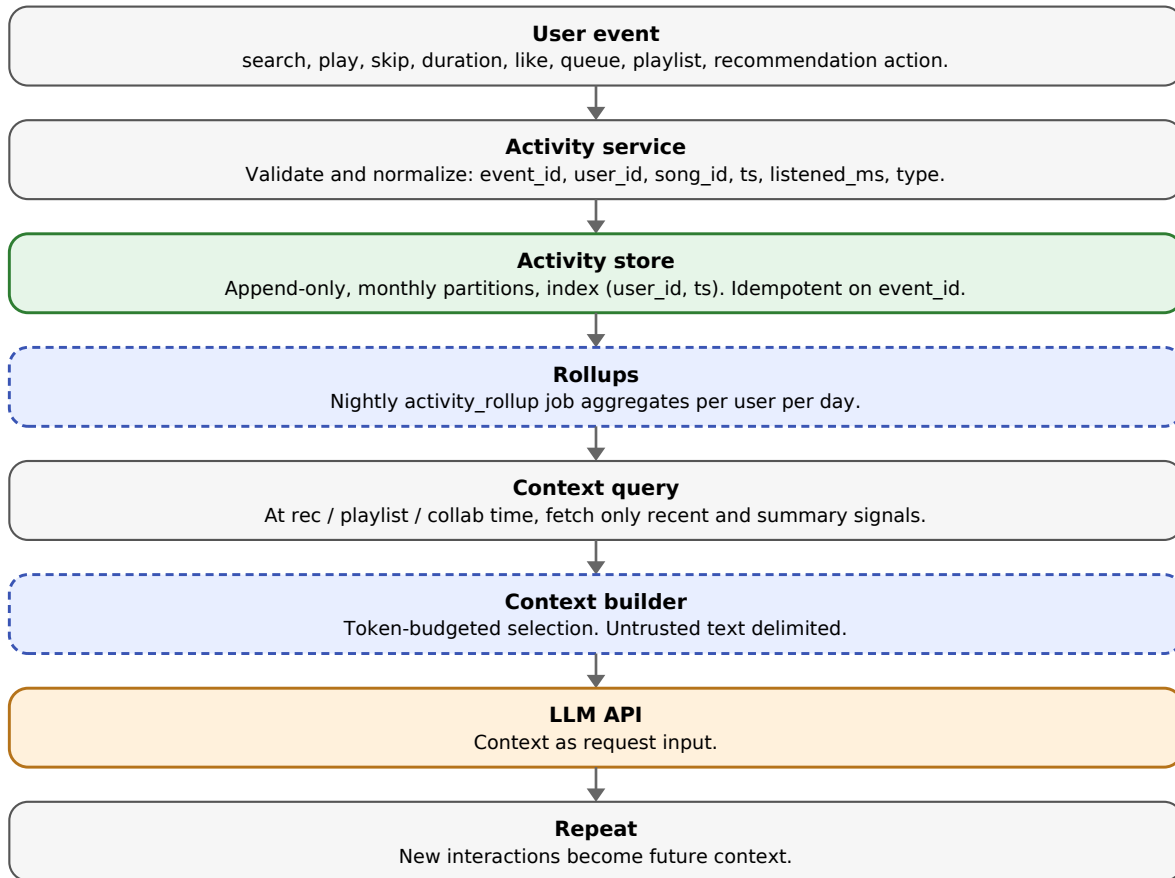
1. **Browser:** User selects download.
2. **API / Auth:** Authenticate. Song must be READY, otherwise 202 and resolve first.
3. **Token:** Mint short-lived signed token with dl=1, exp ≤10 min.
4. **Delivery:** Presigned object URL, or GET /stream/{token}?dl=1 with Content-Disposition: attachment and Range support.
5. **Client:** Browser downloads the original audio format.
6. **Audit:** Log a download event.

Gaps filled (decisions in this revision)

- No transcoding pipeline. The download is the stored file as-is. Download events are part of the activity taxonomy.

13. Activity Logging & LLM Context Preparation

Personalization without model training: events accumulate and are selectively assembled into LLM context.



Backend execution

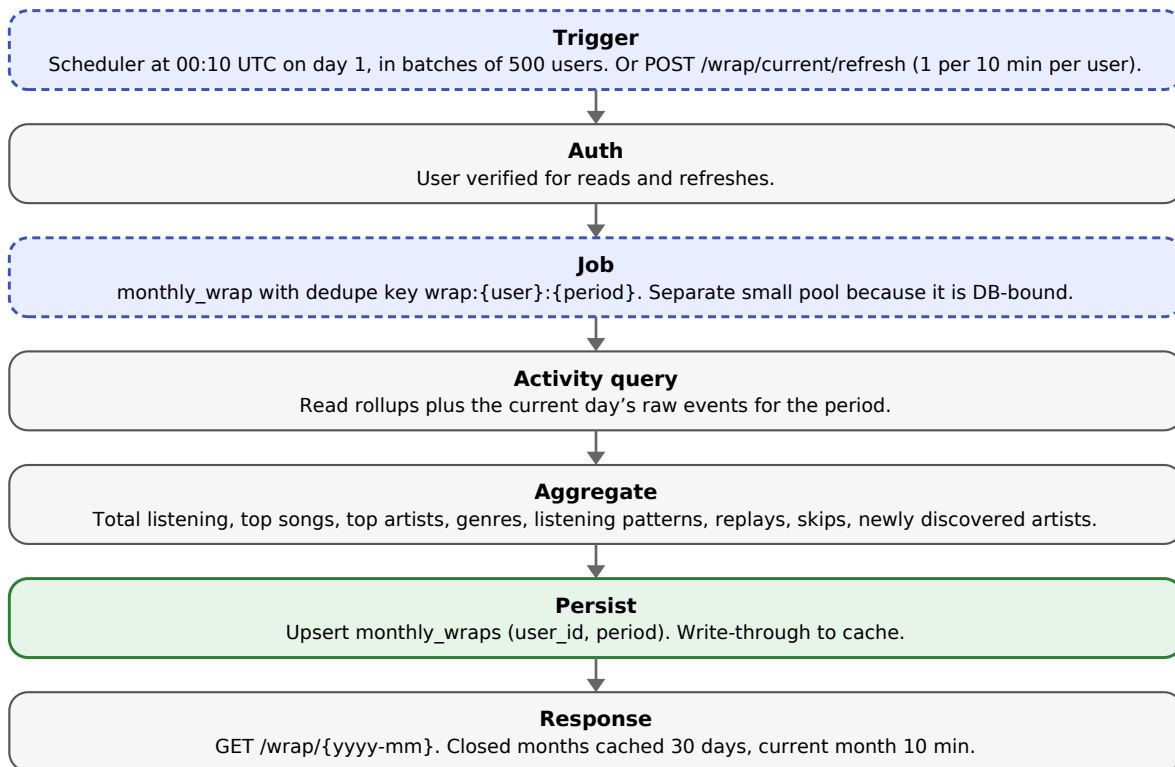
1. **User event:** search, play, skip, duration, like, queue, playlist, recommendation action.
2. **Activity service:** Validate and normalize: event_id, user_id, song_id, ts, listened_ms, type.
3. **Activity store:** Append-only, monthly partitions, index (user_id, ts). Idempotent on event_id.
4. **Rollups:** Nightly activity_rollup job aggregates per user per day.
5. **Context query:** At rec / playlist / collab time, fetch only recent and summary signals.
6. **Context builder:** Token-budgeted selection. Untrusted text delimited.
7. **LLM API:** Context as request input.
8. **Repeat:** New interactions become future context.

Gaps filled (decisions in this revision)

- Retention: raw events 24 months, then only rollups. Partitions are created monthly by the scheduler and dropped after expiry.

14. Monthly Wrap

Computed by a scheduled, idempotent job and served from cache.



Backend execution

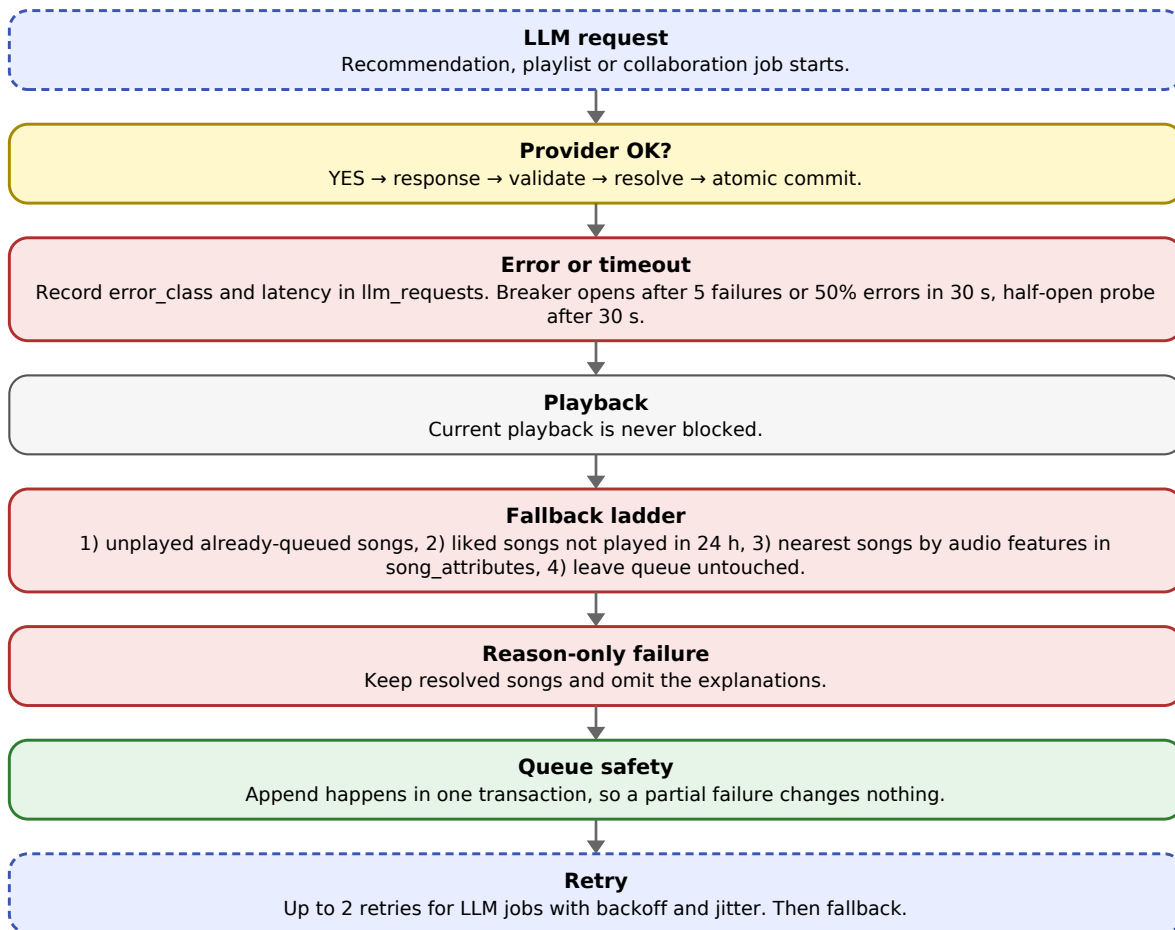
1. **Trigger:** Scheduler at 00:10 UTC on day 1, in batches of 500 users. Or POST /wrap/current/refresh (1 per 10 min per user).
2. **Auth:** User verified for reads and refreshes.
3. **Job:** monthly_wrap with dedupe key wrap:{user}:{period}. Separate small pool because it is DB-bound.
4. **Activity query:** Read rollups plus the current day's raw events for the period.
5. **Aggregate:** Total listening, top songs, top artists, genres, listening patterns, replays, skips, newly discovered artists.
6. **Persist:** Upsert monthly_wraps (user_id, period). Write-through to cache.
7. **Response:** GET /wrap/{yyyy-mm}. Closed months cached 30 days, current month 10 min.

Gaps filled (decisions in this revision)

- Counted play = listened ≥ 30 s. Replay = a song with ≥ 2 counted plays. Skip = explicit skip or < 30 s.
- Newly discovered artist = 0 plays in the previous 12 months and ≥ 1 counted play in the period.
- Double computation is impossible: unique key plus job dedupe.

15. LLM Failure, Fallback & Graceful Degradation

LLM failures never break playback or corrupt a queue.



Backend execution

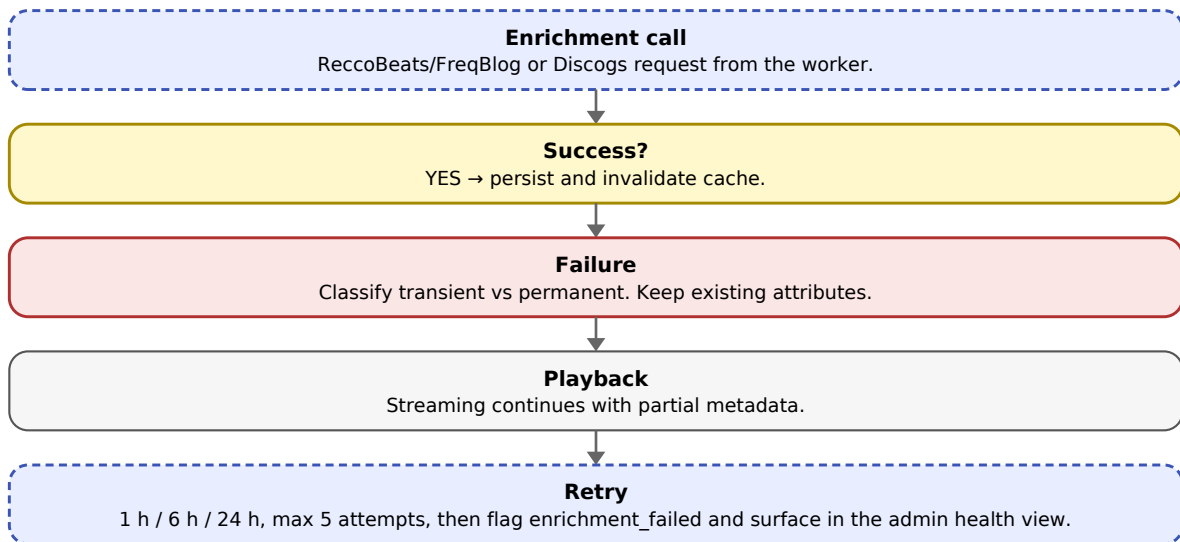
1. **LLM request:** Recommendation, playlist or collaboration job starts.
2. **Provider OK?:** YES → response → validate → resolve → atomic commit.
3. **Error or timeout:** Record error_class and latency in llm_requests. Breaker opens after 5 failures or 50% errors in 30 s, half-open probe after 30 s.
4. **Playback:** Current playback is never blocked.
5. **Fallback ladder:** 1) unplayed already-queued songs, 2) liked songs not played in 24 h, 3) nearest songs by audio features in song_attributes, 4) leave queue untouched.
6. **Reason-only failure:** Keep resolved songs and omit the explanations.
7. **Queue safety:** Append happens in one transaction, so a partial failure changes nothing.
8. **Retry:** Up to 2 retries for LLM jobs with backoff and jitter. Then fallback.

Gaps filled (decisions in this revision)

- The deterministic fallback policy and retry parameters are now fixed.
- HTTP mapping: 502/503 with Retry-After for dependency failure, 504 for upstream timeout. Direct synchronous endpoints are not LLM-bound.

16. External Enrichment Failure

Non-critical providers fail independently from playback.



Backend execution

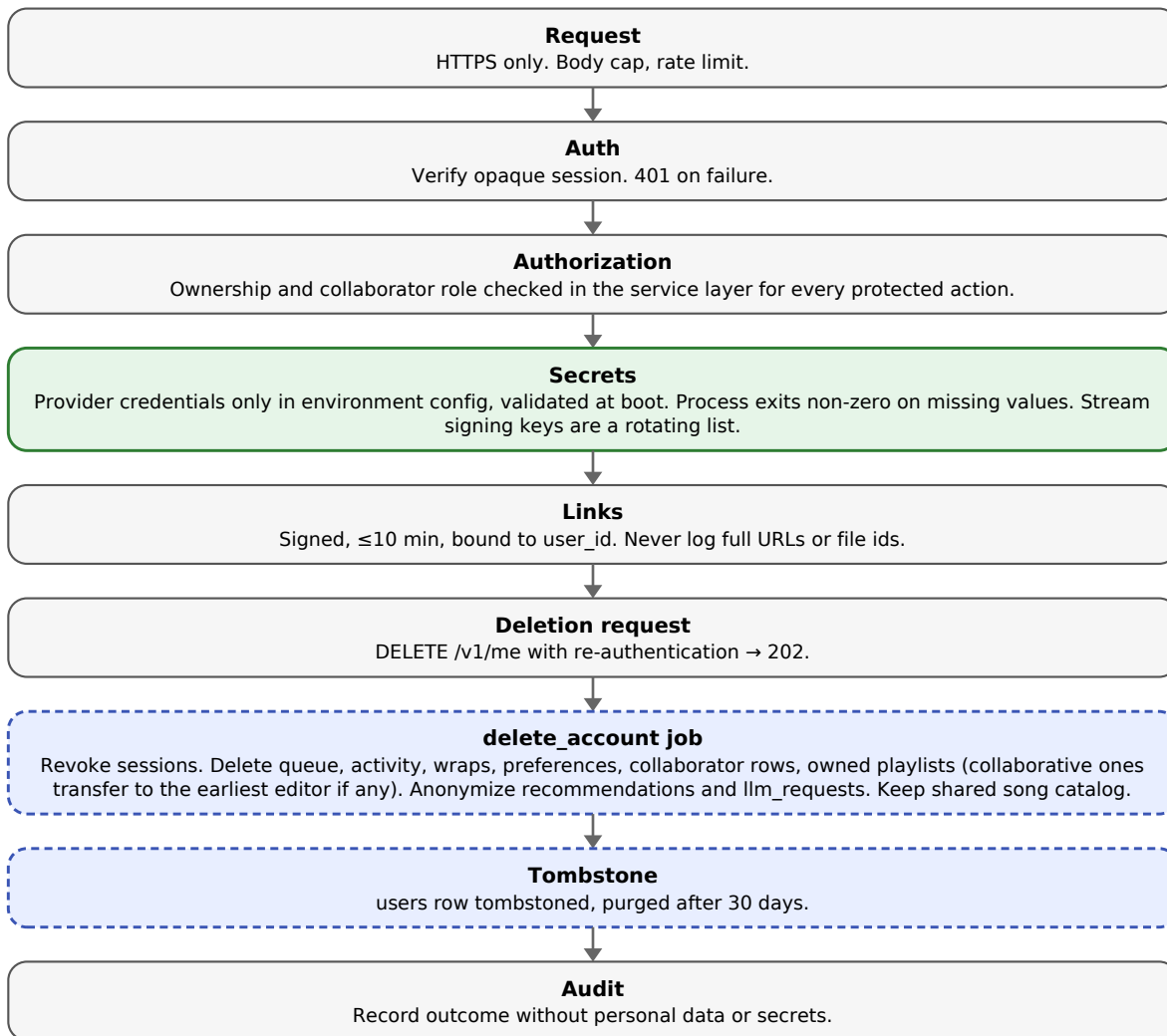
1. **Enrichment call:** ReccoBeats/FreqBlog or Discogs request from the worker.
2. **Success?:** YES → persist and invalidate cache.
3. **Failure:** Classify transient vs permanent. Keep existing attributes.
4. **Playback:** Streaming continues with partial metadata.
5. **Retry:** 1 h / 6 h / 24 h, max 5 attempts, then flag `enrichment_failed` and surface in the admin health view.

Gaps filled (decisions in this revision)

- Rate limits (Discogs 50/min) are enforced client-side with a token bucket so retries do not self-inflict 429s.

17. Security, Authorization & Account Deletion

Cross-cutting controls for user data, collaboration, credentials and deletion.



Backend execution

1. **Request**: HTTPS only. Body cap, rate limit.
2. **Auth**: Verify opaque session. 401 on failure.
3. **Authorization**: Ownership and collaborator role checked in the service layer for every protected action.
4. **Secrets**: Provider credentials only in environment config, validated at boot. Process exits non-zero on missing values. Stream signing keys are a rotating list.
5. **Links**: Signed, ≤10 min, bound to user_id. Never log full URLs or file ids.
6. **Deletion request**: DELETE /v1/me with re-authentication → 202.
7. **delete_account job**: Revoke sessions. Delete queue, activity, wraps, preferences, collaborator rows, owned playlists (collaborative ones transfer to the earliest editor if any). Anonymize recommendations and llm_requests. Keep shared song catalog.
8. **Tombstone**: users row tombstoned, purged after 30 days.
9. **Audit**: Record outcome without personal data or secrets.

Gaps filled (decisions in this revision)

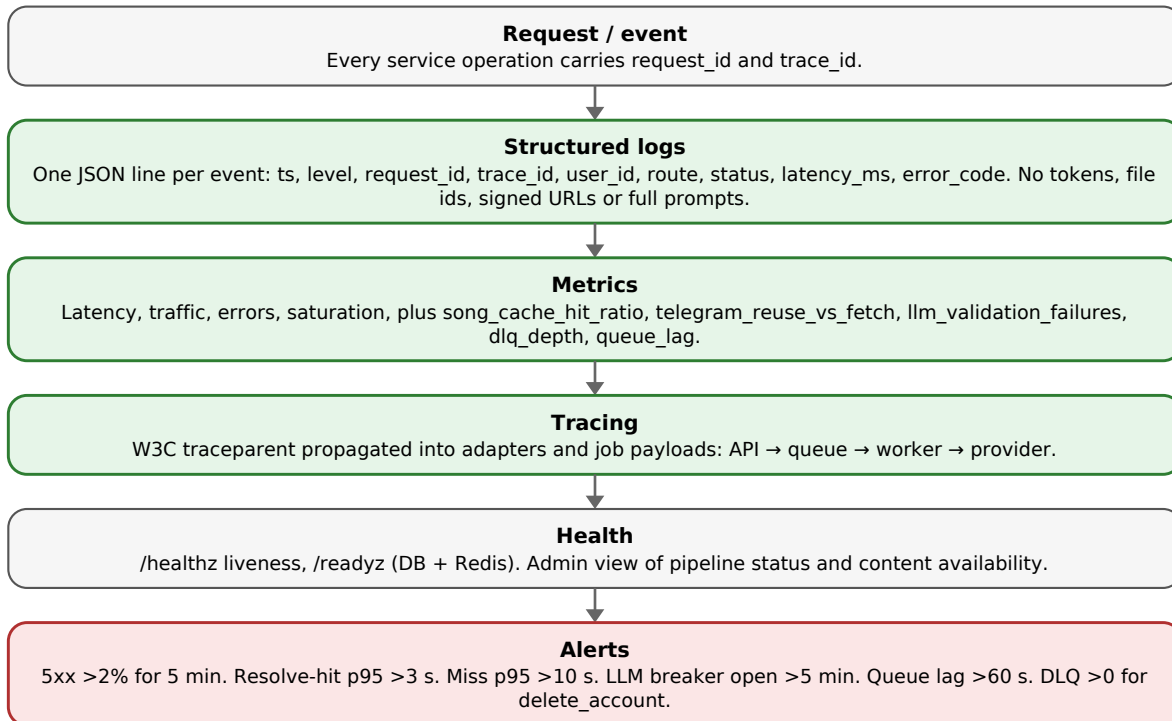
- Deletion cascade and retention are defined. Deletion completes within 30 days, normally within minutes.
- Supabase service-role key remains server-side. Row-level security is enabled as defense in depth.

Still open

- Confirm 30-day tombstone period with legal.

18. Observability & Operational Monitoring

Telemetry for storage efficiency, LLM reliability, and core playback health.



Backend execution

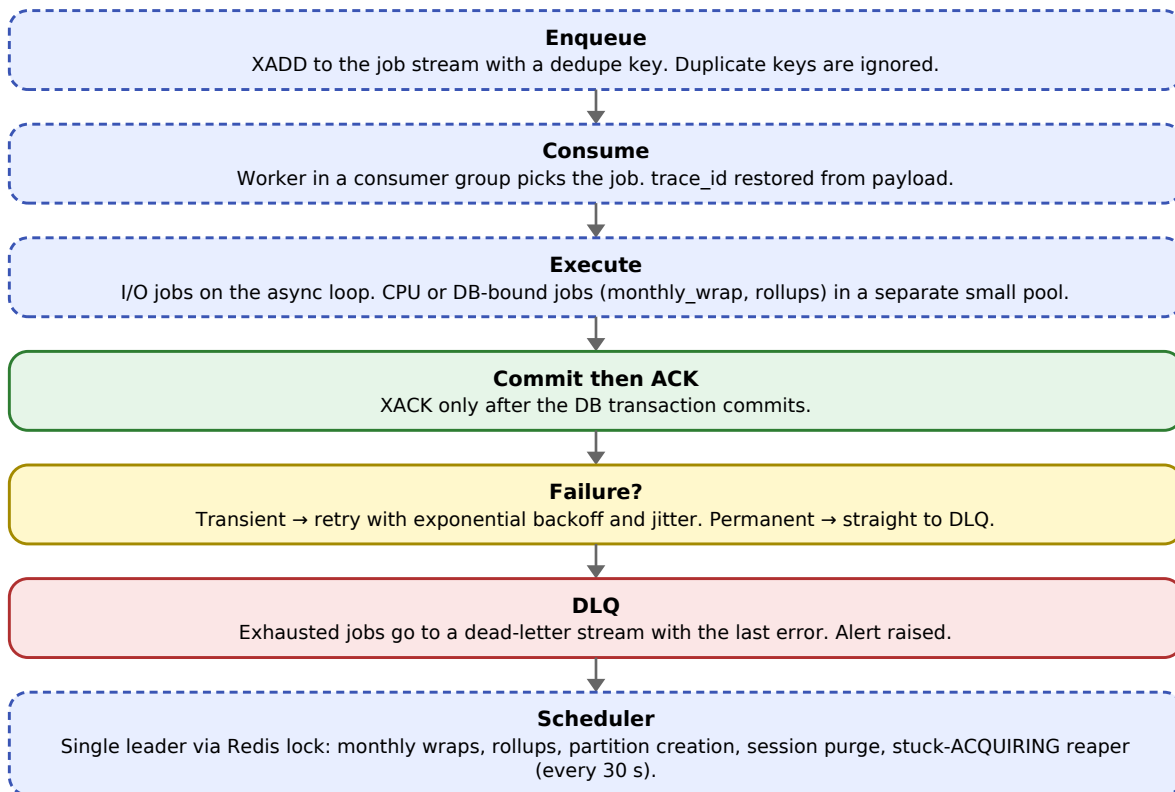
1. **Request / event:** Every service operation carries request_id and trace_id.
2. **Structured logs:** One JSON line per event: ts, level, request_id, trace_id, user_id, route, status, latency_ms, error_code. No tokens, file ids, signed URLs or full prompts.
3. **Metrics:** Latency, traffic, errors, saturation, plus song_cache_hit_ratio, telegram_reuse_vs_fetch, llm_validation_failures, dlq_depth, queue_lag.
4. **Tracing:** W3C traceparent propagated into adapters and job payloads: API → queue → worker → provider.
5. **Health:** /healthz liveness, /readyz (DB + Redis). Admin view of pipeline status and content availability.
6. **Alerts:** 5xx >2% for 5 min. Resolve-hit p95 >3 s. Miss p95 >10 s. LLM breaker open >5 min. Queue lag >60 s. DLQ >0 for delete_account.

Gaps filled (decisions in this revision)

- Stack proposed: OpenTelemetry, Prometheus, Grafana. Thresholds above are starting values to tune from production data.

19. Background Jobs & Scheduler (new)

Common contract for every asynchronous workload.



Backend execution

1. **Enqueue:** XADD to the job stream with a dedupe key. Duplicate keys are ignored.
2. **Consume:** Worker in a consumer group picks the job. trace_id restored from payload.
3. **Execute:** I/O jobs on the async loop. CPU or DB-bound jobs (monthly_wrap, rollups) in a separate small pool.
4. **Commit then ACK:** XACK only after the DB transaction commits.
5. **Failure?:** Transient → retry with exponential backoff and jitter. Permanent → straight to DLQ.
6. **DLQ:** Exhausted jobs go to a dead-letter stream with the last error. Alert raised.
7. **Scheduler:** Single leader via Redis lock: monthly wraps, rollups, partition creation, session purge, stuck-ACQUIRING reaper (every 30 s).

Gaps filled (decisions in this revision)

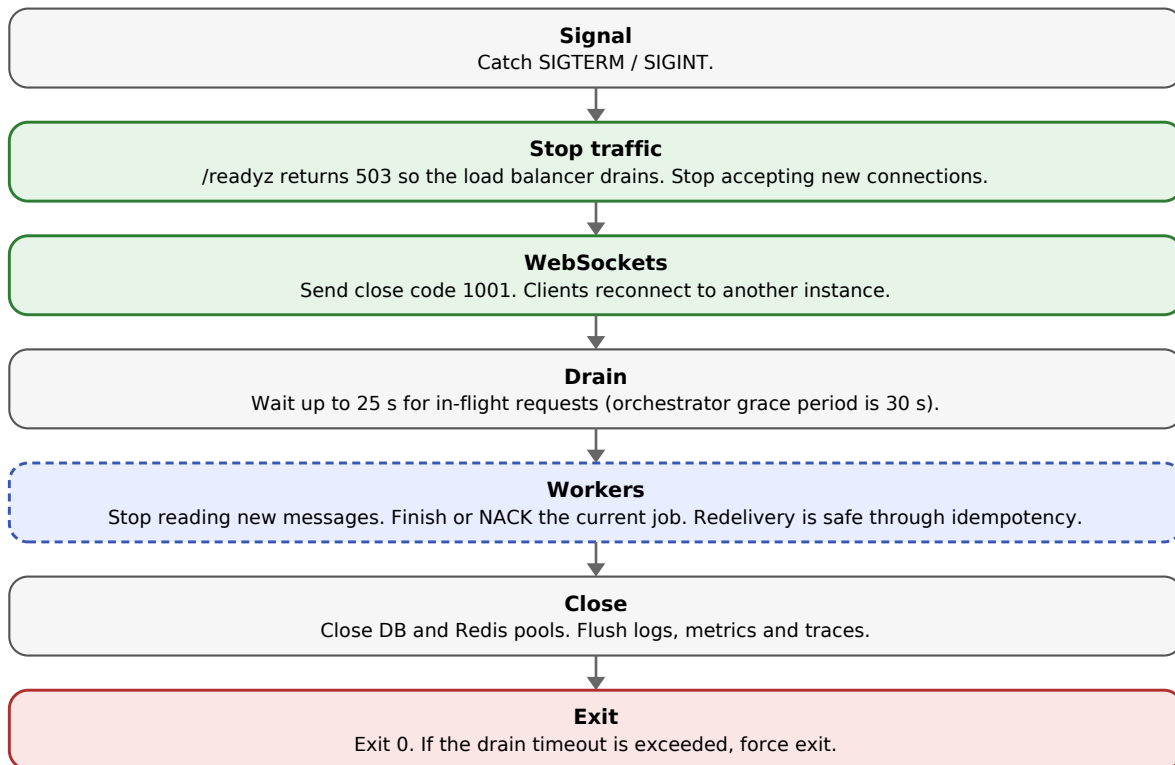
- Semantics are at-least-once with idempotent handlers. See Appendix A for the job catalog.

Still open

- Redis Streams vs a Postgres SKIP LOCKED queue if a single datastore is preferred.

20. Graceful Shutdown & Deploy (new)

On SIGTERM the system finishes what it started and drops nothing.



Backend execution

1. **Signal:** Catch SIGTERM / SIGINT.
2. **Stop traffic:** /readyz returns 503 so the load balancer drains. Stop accepting new connections.
3. **WebSockets:** Send close code 1001. Clients reconnect to another instance.
4. **Drain:** Wait up to 25 s for in-flight requests (orchestrator grace period is 30 s).
5. **Workers:** Stop reading new messages. Finish or NACK the current job. Redelivery is safe through idempotency.
6. **Close:** Close DB and Redis pools. Flush logs, metrics and traces.
7. **Exit:** Exit 0. If the drain timeout is exceeded, force exit.

Gaps filled (decisions in this revision)

- Rolling deploys cannot lose jobs, break streams mid-request, or leave half-written playlists.

Appendix A. Job Catalog

All jobs run on Redis Streams consumer groups. A job is ACKed only after its DB transaction commits, so a crash re-delivers it and idempotency keys make that safe.

Job	Payload	Dedupe key	Retry / notes
acquire_song	song_id	song:{id}	I/O pool (8). 4 retries expo+jitter, DLQ. Reaper requeues after 2 min.
enrich_song	song_id	enrich:{id}:{v}	Token bucket (Discogs 50/min). 1 h / 6 h / 24 h, max 5.
generate_recs	user_id, anchor_song_id, trigger_id	rec:{user}:{trigger}	20 s LLM timeout, 2 retries, then fallback ladder.
generate_playlist	user_id, seed_dto	Idempotency-Key	2 retries. Draft stored 1 h.
collab_suggest	playlist_id, version	suggest:{id}:{version//10}	1 retry. Coalesces bursts.
monthly_wrap	user_id, period	wrap:{user}:{period}	DB-bound pool. 3 retries.
activity_rollup	day	rollup:{day}	Nightly. 3 retries.
delete_account	user_id	del:{user}	5 retries. DLQ triggers alert.

Appendix B. Error → HTTP Status Matrix

Class	Examples	Status	Notes
Validation	Malformed JSON, field out of range	400 / 422	Structured field errors. No retry.
Authentication	Missing or expired session	401	Cookie cleared. Re-login.
Authorization	Viewer tries to edit	403 (or 404)	404 when existence must stay hidden.
Conflict	Stale If-Match, duplicate	409 / 412	Refetch and retry.
Precondition missing	No If-Match header	428	Add header.
Rate limit	Bucket empty	429	Retry-After header.
Not ready	Song ACQUIRING	202	Location header, poll or SSE.
Dependency down	LLM, Song API breaker open	502 / 503	Retry-After. Playback unaffected.
Dependency timeout	Upstream over deadline	504	Retry.
Fatal	Unhandled bug	500	Generic body with request_id. Stack trace in logs only.

Appendix C. Still Open (needs a named owner)

- Audio delivery: confirm object storage with Telegram as backup (recommended) versus Telegram-only with a signed streaming endpoint.
- Legal review: sourcing audio from the Song API, re-hosting it, and using Telegram as storage.
- LLM provider, model and exact prompt text (schema and limits are defined).
- Search provider (MusicBrainz or the Song API search) and lyrics provider (LRCLIB proposed) terms of use.
- Queue technology: Redis Streams (proposed) versus Postgres SKIP LOCKED.
- Account-deletion tombstone period (30 days proposed) and activity retention (24 months proposed).
- Whether email/password login is added after MVP.